FIELD GUIDE 03 / IMPLEMENTATION TO EVIDENCE

IdentityForge

The hiring asset upgrade

Actual AD lifecycle tooling. A real-Entra-ready SSO client. Operational decisions you can defend.

IMPLEMENTED IS NOT THE SAME AS LIVE-VERIFIED

The AD planner and executor pass offline checks. The SSO client passes local tests/build and an HTTP delivery check. No AD endpoint or lab tenant is available yet; live directory execution and tenant sign-in are explicitly pending.

Start with the AD preview checkpoint. Complete its verification gate before applying changes. Use the SSO runbook when a suitable isolated tenant becomes available.

Contents

Getting started / AD lifecycle checkpoints / Entra SSO setup and break-fix / provisioning investigation / access certification / privileged incident response / portfolio scope and verification.

START HERE

Build the evidence, not just the narrative

This upgrade adds executable AD lifecycle tooling, an Entra SSO client, operational drills and a curated source-bundle exporter. It builds on the standalone lab without changing your saved simulation progress.

Start with one checkpoint

1. If you do not yet have an isolated Windows Server AD domain, use the setup PDF, then advanced/CHECKPOINT-01.md. Nothing here automatically installs a DC or joins your Windows host to a domain.

2. Read advanced/lifecycle/README.md and complete its first checkpoint: validate the lab and preview the joiner CSV. Do not use -Apply before inspecting the plan.

3. Continue independently through joiners, approved activation, mover/leaver, failure/recovery and convergence. Keep each run ID and actual verification evidence.

4. Use operational/Export-AccessReview.ps1 and ACCESS-REVIEW-DECISION.md to record a certification decision. Complete PROVISIONING-FAILURE.md and PRIVILEGED-INCIDENT.md with clearly labeled evidence.

5. For real SSO, use sso/README.md. Since no tenant is currently available, local code readiness can be verified but live integration remains pending. There is no fake sign-in or automatic tenant purchase.

6. Fill portfolio/README.md only with accomplishments you actually verified. Use portfolio/Export-Portfolio.ps1 to package reviewed source for your portfolio.

Offline checks available now

From the project root:

```
powershell.exe -NoProfile -File .\advanced\lifecycle\tests\Planner.Tests.ps1
powershell.exe -NoProfile -File .\operational\tests\Review.Tests.ps1
```

In the sso folder:

```
npm ci
npm test
npm run build
npm run dev
```

The local SSO page explains configuration requirements and leaves sign-in unavailable until a real lab tenant and application ID are supplied. The runner performs AD writes only when invoked explicitly against your named domain/DC with -Apply. Initial activation is separate and prompts for a SecureString password.

Strong interview proof

Show a before state, your reviewed change plan, the exact operation/result log, independently verified after state and an example of failure/recovery. Explain which access is managed by birthright policy and which requires owner approval. Distinguish disabling an account from terminating sessions and revoking cloud tokens.

Joiners, movers, leavers and recovery

This track runs actual AD management cmdlets. It is implemented locally, tested offline and **not yet verified against a live lab domain**. Use a dedicated disposable domain with the IAM-Training tree from advanced/01-Build-ADLab.ps1. No cloud tenant is required for this track.

Checkpoint 1 - prerequisites and preview

Copy this project to your isolated Windows Server lab VM or a domain management machine. Use Windows PowerShell 5.1 with the ActiveDirectory module and a delegated lab administrative account. Do not rerun the original foundation builder after moving lab-alex: it deliberately detects a different OU as a collision. Lifecycle reruns are the repeatable workflow from here.

Open PowerShell **in this lifecycle folder**. Substitute your actual lab values:

```
$dc = 'dc01.iamlab.test'
$domain = 'iamlab.test'
.\Invoke-ADLifecycle.ps1 -Server $dc -ExpectedDomain $domain `
    -CsvPath .\data\01-joiners.csv
```

No -Apply means preview. The script reads AD, validates the whole feed and creates a unique evidence folder; it does not mutate AD. -WhatIf also prevents AD writes, even if combined with -Apply. Inspect plan.csv and the console. Confirm every target is synthetic and within IAM-Training. Unknown departments, duplicate HR/directory IDs, identity-name mismatches, privileged/protected accounts, nonstandard primary groups and out-of-scope matches stop preflight.

Gate: inspect the plan and initial evidence. **Do only this checkpoint on your first attempt.** The remaining checkpoints are available below for self-paced use after each gate passes.

Checkpoint 2 - joiner provisioning

After reviewing the preview:

```
.\Invoke-ADLifecycle.ps1 -Server $dc -ExpectedDomain $domain `
    -CsvPath .\data\01-joiners.csv -Apply
```

New-ADUser creates missing users disabled without embedded passwords. Set-ADUser reconciles HR attributes; groups use Add-ADGroupMember/Remove-ADGroupMember. The existing foundation user is initially disabled too. FollowUpRequired (exit 2) is expected until explicit activation: provisioning is not sign-in authorization. Department/OU names and birthright groups are fixed in Get-IamBirthright, not supplied as arbitrary distinguished names in HR.

The script uses EmployeeID as the HR lookup key and retains ObjectGUID for existing identities during a run. It does not implement account renames or change a cloud sourceAnchor. UserPrincipalName is generated with the local lab domain suffix; it is not proof of cloud usability or synchronization. mailNickname is not set automatically because a plain AD schema may not expose it.

Gate: four synthetic users exist with the right groups/attributes, new users are disabled, and logs match the actual outcomes.

Checkpoint 3 - controlled initial activation

Verify the HR status is Active, initial access matches the approved role, and a fictional training change ticket is approved. Preview, then activate **one account at a time**:

```
.\Enable-LabJoiner.ps1 -Server $dc -ExpectedDomain $domain `
    -EmployeeID LAB-001 -Ticket CHG-LAB-100
.\Enable-LabJoiner.ps1 -Server $dc -ExpectedDomain $domain `
    -EmployeeID LAB-001 -Ticket CHG-LAB-100 -Apply
```

The helper prompts securely for a unique lab password, requires change at next sign-in, then enables and rereads the account. Do not store passwords in CSV or audit notes. Repeat separately for LAB-002, LAB-003 and LAB-004 using approved tickets. It refuses expired/Terminated/protected accounts and does not reset an already-enabled account's password. A ticket string is training evidence, not a real approval engine. Domain password policies still apply.

Rerun the joiner lifecycle preview after activation: it should show no remaining operations. Preserve the evidence proving this convergence. A disabled existing account is never implicitly enabled by an Active HR row; rehire needs a separate process.

Checkpoint 4 - transfer and termination

```
.\Invoke-ADLifecycle.ps1 -Server $dc -ExpectedDomain $domain `
    -CsvPath .\data\02-mover-leaver.csv
.\Invoke-ADLifecycle.ps1 -Server $dc -ExpectedDomain $domain `
    -CsvPath .\data\02-mover-leaver.csv -Apply
```

Expected for Alex: remove LAB-Support-Tickets before granting LAB-Engineering-Tools, set Department Engineering and move to the Engineering OU. Explicitly approved exceptional LAB groups remain untouched on movers; only the five managed birthright groups are reconciled.

Expected for Taylor: Disable-ADAccount first, Set-ADAccountExpiration at the run's UTC time, remove all removable direct LAB-group memberships including exceptions, then Move-ADObject to Terminated. Domain Users primary membership is preserved. Direct memberships outside recognized lab groups are reported as FollowUpRequired, not modified or described as removed. Existing logon sessions/tickets, nested or cross-system access, app sessions and cloud tokens are not revoked by this script. Lab groups nested outside the approved lab group tree block preflight.

Gate: summary.csv has Verified for completed rows, remaining-plan.csv has no outstanding mutation for those rows, and AD inspection confirms the actual groups/OU/disabled/expiration state. After replication, test access through a fresh sign-in on an optional isolated client; do not infer session termination from group removal alone.

Checkpoint 5 - failure, recovery and repeatability

1. Preview data/03-invalid-feed.csv. Expect a terminating validation error before any AD write. Correct UNKNOWN to HR in a copy and retry. Never edit the original fixture when keeping before/after evidence.

2. Offline executor tests inject a directory-operation failure. Read tests/Planner.Tests.ps1 and observe that subsequent changes for the same identity stop while independent identities can continue. This is an **offline injected failure**, not proof of a real connector outage.

3. For a live failure exercise, use a separate delegated lab operator missing the relevant group-write permission. Do not remove production permissions or break the DC. Capture the failed operation and subsequent skipped work, then restore the approved lab delegation and rerun.

4. Re-preview the mover/leaver feed. Verified rows should generate no operations. Failed rows should converge after the underlying cause is corrected. There is no fictitious rollback: earlier successful operations remain and are reconciled on retry.

Gate: write an incident note using operational/PROVISIONING-FAILURE.md. Include failed identity/operation, ticket/run ID, actual state, temporary risk, cause, approved correction, retry result and evidence. Exit 2 means failure or follow-up, not success.

Evidence and audit behavior

Each run gets evidence/ad-lifecycle/<run-guid> containing plan.csv, before.json, after.json, remaining-plan.csv, summary.csv and audit.jsonl. Validation failures before a run folder exists are terminating console errors: capture a transcript for the invalid-input exercise. Directory errors and attempted operations are logged with intent before mutation and success/failure afterwards. Audit write failure stops further writes. Per-identity execution stops after the first failed operation, preserving earlier successes. Local file logs are not tamper-resistant or centralized security telemetry.

A shared lock prevents simultaneous runs using the same OutputRoot. Keep OutputRoot consistent across operators; changing it bypasses that local lock. The lock does not coordinate unrelated administrative tools. No multi-object AD transaction is implied. Check replication on other DCs separately.

Raw snapshots contain lab directory identifiers. Keep them private until reviewed; the portfolio exporter excludes them by default. Do not publish passwords, Kerberos material, tokens or tenant secrets.

Local verification

```
powershell.exe -NoProfile -File .\tests\Planner.Tests.ps1
```

This tests planner/executor behavior without an AD server. Live AD checks are a separate requirement. Microsoft references: New-ADUser, Set-ADUser, Remove-ADGroupMember, Disable-ADAccount, Set-ADAccountExpiration.

Entra SSO setup and break-fix

This is a working browser application using Microsoft MSAL, not a fake IdP. It needs an isolated Entra workforce lab tenant, a test user and your own app registration. **You currently have no tenant.** Code/tests do not establish live sign-in success, and no tenant was provisioned for you.

Checkpoint 1 - obtain a suitable lab tenant

Use an isolated tenant you are entitled to administer. Tenant/trial availability and signup eligibility vary; check Microsoft's current account/tenant offerings. Do not connect an employer's tenant or buy a subscription merely to run an unreviewed exercise. You need permission to register the application and sign in a test user. This basic sign-in app does not require Conditional Access configuration. Device/risk policies, their licensing, and hybrid sync are separate extensions.

If no tenant is available, run npm test and review the app locally. Record status as Prepared, not Integrated. There is no mock-success button. Do not paste passwords, client secrets or tokens into this chat or source files.

Checkpoint 2 - register an isolated SPA

1. In the Entra admin center, select your lab directory and open App registrations > New registration.

2. Name it IdentityForge SSO Lab. Select accounts in this organizational directory only.

3. Configure the **Single-page application** platform redirect URI as `http://localhost:5173/` exactly. Do not enable implicit grant or create a client secret.

4. Copy Application (client) ID and Directory (tenant) ID. These IDs are configuration identifiers, not passwords; still keep tenant details out of public screenshots unless intentionally disclosed.

5. Under API permissions, no business API is called by this app. Remove an automatically added Microsoft Graph User.Read permission if present and unused. The app requests only openid and profile for sign-in; do not grant directory-write permissions.

6. Check Enterprise applications and user assignment requirements. If assignment is required, assign only your lab test user through an authorized lab admin. Keep your tenant's existing safety controls enabled.

Gate: verify the intended tenant, single-tenant registration, SPA platform and exact redirect URI. Record the configuration privately without a client secret.

Checkpoint 3 - run locally

Use Node.js 22.12 or later. In the sso folder:

```
npm ci
Copy-Item .\public\config.example.json .\public\config.local.json
notepad .\public\config.local.json
npm test
npm run build
npm run dev
```

If the package lock is absent, use npm install once to generate it. Open **http://localhost:5173/**, not an IP address, another port or file://. The dev server binds only to localhost. config.local.json is ignored by Git but is served to the local browser; it must contain no secret. Do not publish the dev server. Stop with Ctrl+C.

Sign in using the explicit Sign in button. MSAL performs the authorization-code/PKCE browser flow; redirect handling runs once on page load. The app verifies its expected claim contract after MSAL returns. It does not implement a protected backend API and it does not independently verify JWT signatures. A production backend must validate its own access tokens and enforce authorization.

Gate: successful MSAL response, Healthy mapping passes, and a matching successful application sign-in exists in your Entra lab sign-in logs. Test a second sign-in with the existing lab IdP session and record the observed SSO experience; consent/MFA/session policies may still require interaction. Do not claim passwordless or MFA enforcement solely from this app's success.

Checkpoint 4 - deliberate application mapping failure

1. With a verified Healthy session, download baseline event evidence.

2. Switch the application contract to Broken. It now incorrectly requires legacy_employee_number, a claim not configured in this lab.

3. Recheck. Expect MAPPING_CLAIM_MISSING. Authentication may have succeeded while application mapping fails: these are different layers.

4. Capture the rejection and compare the app contract with documented ID token claims. The example uses a fictional legacy field to reproduce a contract mismatch; it does not pretend Entra emits AD immutableID or UPN in every token.

5. Observe that the app **does not** automatically redirect into another login. Reauthentication cannot repair the missing mapping. In a poorly built app this unconditional retry behavior can cause a loop.

6. Restore Healthy and recheck the same session. Download the recovery evidence and write a root-cause note. Do not edit the real token, bypass tenant checks, change sourceAnchor/immutableID or disable MFA to fix the exercise.

Gate: observed baseline, rejection and recovery; no automatic retry loop; root cause names the application mapping contract. This is a fault deliberately introduced at the **application layer**, not a tenant claims-mapping deployment or a live AD-to-cloud synchronization test.

Optional independent registration fault

Only on this disposable registration, remove its SPA redirect URI after exporting/saving the correct setting. Start a fresh sign-in; expect a redirect mismatch such as AADSTS50011. Record actual error code and matching sign-in logs, then restore the exact URI and retest. Do not make changes to another application, broad tenant settings or real users. This optional exercise is not complete until you observe and record both failure and recovery.

Verify the scope of your SSO result

The event download contains timestamps and fixed event codes, not tokens, names or raw claims. Browser event history is editable and is not a security audit authority; corroborate it with tenant logs and your own screenshots. Never include HAR files without expert review: they can contain tokens. Identity uses tid + oid; preferred_username and email are mutable display information, not authorization keys. Domain-joined AD sign-in and this cloud sign-in remain separate unless you later configure and prove a supported hybrid integration.

References checked October 18, 2026:

- MSAL initialization and redirect handling
- MSAL user sign-in
- Register an application
- ID token claims

This implementation selects Entra ID. An Okta integration is not included or claimed.

OPERATIONAL JUDGMENT / WORKSHEET

Investigate a provisioning failure

Evidence type: [actual AD lab / offline injected failure]. Date: [UTC]. Investigator: [name]. Change ticket/run ID: [reference].

Trigger and impact

- Expected HR change and immutable employee ID:

- Failed operation, target system and observed error:

- Last successful operation before failure:

- Current account enabled/disabled, OU and actual groups:

- Exposure or business impact while incomplete:

Investigation and decision

1. Validate the authoritative HR record and correlate EmployeeID, account and ObjectGUID.

2. Check required OU/group existence, delegated permissions, selected DC, DNS/connectivity and replication. Distinguish invalid input from access denied or transient infrastructure failure.

3. Inspect the specific run's intent/result events and remaining-plan.csv. Do not infer rollback.

4. Identify temporary containment and the owner who approves it. Revocation failure may require urgent owner/incident escalation; never silently leave high-risk old access active.

5. Record the smallest approved correction; avoid granting Domain Admin just to make a connector work.

Root cause: [specific finding, not simply script failed]. Correction: [change/ticket]. Retry and reconciliation result: [observed]. Independent verification: [evidence]. Unresolved items/owner/due date: [details].

Completion gate

The target directory confirms desired state, remaining operations are understood, partial changes are accounted for, and evidence supports the reported result. Offline tests are labeled offline. Report repeatability, not invented success percentages.

Document an access review decision

Review ID: [reference]. Evidence source/run ID: [reference]. Scope: [actual AD snapshot / offline fixture].

Identity/employee ID: [synthetic identity]. Resource/group: [exact LAB group]. Business owner/reviewer: [name]. Review date and deadline: [UTC].

Finding: [e.g., LAB-Payment-Creator plus LAB-Payment-Approver]. Risk: the same identity can create and approve a payment in this policy model. These lab groups have no actual payment-system permissions unless you separately assign them.

Decision: [retain / revoke / time-bound exception]. Business justification: [specific task]. Approved change ticket: [reference]. Exception expiry, compensating controls and accountable owner: [required if retained by exception].

Execute an approved revocation

Use a lab administrative account against the explicitly verified lab DC. Resolve the LAB group and user, confirm both are inside IAM-Training, record an approved decision **before** removal, then preview Remove-ADGroupMember with -WhatIf. Apply only the approved group removal; preserve unrelated valid access. Reread direct and relevant nested membership afterwards and save the evidence. Never substitute a production group.

Requested by: [name]. Approved by: [independent owner]. Executed by: [name]. Verified by: [name/evidence]. Actual removal time: [UTC]. Remaining risk: [details].

If the manager refuses: explain the concrete conflict, validate task need, propose narrower access or staged change, document the refusal, route any exception through risk approval with expiry, and escalate overdue high-risk exposure through the governance process. Do not invent approval or close an unresolved finding.

INCIDENT RESPONSE / TABLETOP

Contain privileged identity compromise

This is an operational judgment exercise. Do not execute attack commands or disable real business accounts. Evidence type: [tabletop / actual isolated lab event].

Trigger

Synthetic Windows Security event 4728 (a member added to a security-enabled **global** group):

```
TimeCreated: 2026-10-18T14:05:00Z
EventID: 4728
Computer: DC01.iamlab.test
SubjectUserName: lab-operator
SubjectDomainName: IAMLAB
SubjectLogonId: 0x7A12F
MemberName: CN=lab-alex,OU=Engineering,OU=IAM-Training,DC=iamlab,DC=test
MemberSid: S-1-5-21-111111111-222222222-333333333-1101
TargetUserName: LAB-Tier0-Simulation
TargetDomainName: IAMLAB
TargetSid: S-1-5-21-111111111-222222222-333333333-2201
```

This is a synthetic field rendering, not an exported Windows event. LAB-Tier0-Simulation has no real Tier 0 rights in this package. Compare the acting subject, member and target group with the approved ticket, event time and logon session. A single event proves neither compromise nor all subsequent activity. Event 4732 is for security-enabled local/domain-local groups; 4756 is for universal groups.

Response worksheet

1. Validate severity and event origin; correlate with 4624 sign-in/session evidence, endpoint activity, membership changes and change approvals. Page incident response when compromise is credible.

2. Preserve raw event records and relevant session evidence with timestamps/access controls. Log your containment actions; do not erase evidence.

3. Through approved controls, revoke unauthorized grants, terminate applicable sessions and revoke tokens. Restrict/disable the compromised identity and isolate affected hosts as justified by impact. Account disabling alone does not invalidate every existing ticket or session.

4. Protect continuity and emergency access; coordinate with system owners before broad disruptive action. Rotate compromised credentials and identify dependent services.

5. Scope lateral movement and data exposure, recover from a trusted state, verify containment, monitor and document root cause.

Decision log: [time / observation / action / approver / result]. Assumptions: [known vs unverified]. Recovery criteria: [verifiable requirements]. Lessons: [specific control improvement].

Gate: a reasoned timeline distinguishes facts from hypotheses and explains targeted containment, evidence preservation, session/token handling and business continuity. No actual Domain Admin grant is required for this exercise.

Present a concise, honest portfolio

Problem

HR changes and disconnected access administration cause stale accounts, privilege creep and poorly evidenced remediation. This lab implements repeatable lifecycle tooling and practices evidence-driven incident and access decisions.

Current implementation, not claimed experience

Asset	Implemented	Verification status
Standalone IGA/AM/PAM console	Yes	Local simulation checks; no enterprise controls deployed
CSV-driven AD lifecycle planner and real-cmdlet runner	Yes	Offline automated checks; live AD run pending
Initial AD account activation helper	Yes	Syntax/review; live AD run pending
Entra MSAL sign-in application and mapping break/fix	Yes	Local unit/build checks; no tenant available; live sign-in pending
Access review exporter and decision worksheet	Yes	Offline fixture checks; actual owner decision pending
Privileged compromise investigation	Tabletop	Synthetic event; no real compromise or Domain Admin grant
Hybrid AD-to-cloud synchronization	No	Design concepts only

Use VERIFICATION.md for recorded checks and the private evidence ledger below for your own runs. Do not turn Pending into Verified without an artifact.

Your project summary - fill after doing the work

Business problem: [one sentence]. **Environment:** [actual isolated domain/tenant or simulation]. **What I built:** [specific scripts and integration]. **Failure I investigated:** [actual failure and correction]. **Verification:** [artifact/run IDs and observed outcomes]. **Limitations:** [pending live checks; session/token controls outside scope]. **Next step:** [specific improvement].

Resume wording after implementation but before live execution: "Built a PowerShell AD lifecycle automation lab with CSV validation, managed-group reconciliation, per-identity failure handling and JSONL audit trails; verified planning/execution behavior with offline tests."

After a verified live AD run, add what actually occurred and the isolated scope. After successful cloud sign-in plus break/fix, describe the Entra integration separately. Never describe an offline test as a production deployment or imply employment experience.

Evidence ledger

Stage	Artifact / run ID	Observed result	Evidence kind	Reviewer/date
AD baseline and joiners	[fill]	[fill]	Pending	[fill]
Mover and leaver	[fill]	[fill]	Pending	[fill]
Failure/recovery	[fill]	[fill]	Pending	[fill]
Certification decision	[fill]	[fill]	Pending	[fill]
SSO baseline/break/fix	[fill]	[fill]	Pending	[fill]
Privileged incident judgment	[fill]	[fill]	Tabletop until otherwise proven	[fill]

Share a source bundle

From the project root run `powershell.exe -NoProfile -File .\portfolio\Export-Portfolio.ps1`. It creates a ZIP under output/portfolio-bundles containing allowlisted source, runbooks and verification notes. It intentionally omits tenant configuration, node_modules, working state, raw audit/snapshot files and all unreviewed evidence. It does not publish anything.

Public evidence is a **separate manual selection**: export fixed-code SSO events, review screenshots at full size, remove identifiers and secrets, and add only sanitized copies to your own portfolio. Automated source selection is not a guarantee that future edits contain no secrets; inspect the ZIP before sharing.

A. Implementation milestone - suitable before live AD execution

I expanded IdentityForge from a JSON-based exercise into an AD automation codebase.

The PowerShell workflow validates HR CSV input, plans joiner/mover/leaver changes, reconciles managed birthright groups and records per-operation intent and outcomes. New accounts remain disabled until an explicitly reviewed activation step.

My local tests cover [insert checks you actually ran], including a failed operation that stops further changes for that identity and an audit-loss case that still attempts containment.

The important distinction: these are offline results. Live AD execution is my next verification gate, not an accomplishment I am claiming yet.

Image: a cropped passing-test result and a simple planner -> reviewed changes -> AD -> verification diagram. Label OFFLINE IMPLEMENTATION CHECKS.

#ActiveDirectory #PowerShell #IdentityGovernance #IAM

B. Actual AD lifecycle - only after live verification

I completed the real AD lifecycle stage of my IdentityForge lab in an isolated Windows Server domain.

I used an HR CSV to provision disabled joiners, reviewed and activated the appropriate accounts, moved a synthetic employee from Support to Engineering, and processed a termination. The mover lost old managed access before the new role was granted; the leaver was disabled, expired and moved to the Terminated OU.

I verified [insert observed groups/OU/status results] and preserved a reviewed change plan, audit trail and remaining-state check. My failure/recovery lesson was [actual observation].

This is lab experience, not a production deployment. Session termination, application deprovisioning and cloud token revocation remain separate controls.

Images: actual AD before/after membership and OU view; redacted run summary. Do not publish raw snapshots.

#ActiveDirectory #PowerShell #IdentityLifecycle #LeastPrivilege

C. Real Entra SSO break/fix - only after a live tenant run

I connected the IdentityForge browser application to my isolated Entra lab tenant using MSAL.

After verifying a successful sign-in against the tenant logs, I deliberately introduced a bad application claim requirement. Authentication still succeeded, but the application rejected the missing mapping. Reauthenticating would not fix the contract, so I corrected the application requirement and verified recovery without a redirect loop.

Observed evidence: [baseline / rejection / recovery and tenant-log correlation]. My key lesson: authentication success and application acceptance are different layers.

This demonstrates browser SSO and mapping troubleshooting. It does not claim a protected backend API, hybrid synchronization or Conditional Access enforcement.

Images: sanitized app baseline/rejection/recovery sequence plus a redacted tenant sign-in result. Never publish a token or unreviewed HAR.

#MicrosoftEntra #SSO #IAM #Troubleshooting

D. Operational judgment - name the evidence type

I added an operational decision layer to my IdentityForge portfolio.

In a [tabletop / offline / actual isolated lab] exercise, I investigated [specific provisioning or privileged-access finding], distinguished confirmed facts from assumptions, and documented containment and owner decisions before reporting closure.

My certification decision was [actual decision and rationale]. My recovery check was [observed verification]. The main lesson was [what changed in your approach].

This post describes training evidence at the scope stated above; it is not an account of a production incident.

Image: sanitized timeline and decision record. Label the source type prominently.

#IdentityGovernance #IncidentResponse #IAM #CybersecurityPortfolio

Know exactly what has been checked

Date: October 18, 2026. Environment: local Windows workspace, Windows PowerShell 5.1 and Node.js 22.21.1. No available AD module/domain endpoint or Entra lab tenant has been supplied.

Verification lanes

- AD planner/executor: offline tests cover validation, identity collisions, mover removal order, preserved exceptions, disabled joiners, leaver ordering, convergence, preview no-write behavior, operation failure and audit fail-closed behavior.

- Initial activation: offline injected post-enable logging failure verifies containment occurs independently of logging availability. Actual password/domain-policy and activation behavior still require the lab DC.

- AD orchestration: the real runner executes against a test-only read provider to verify -Apply -WhatIf produces local evidence and never invokes AD mutations. This is not an integration test against Microsoft AD DS.

- Certification export: synthetic input verifies separate identity findings, SoD/stale-leaver detection, pending decisions and duplicate-HR rejection.

- SSO: unit tests verify configuration, tenant/audience/expiry/subject checks, deliberate mapping rejection, lack of automatic retry, safe event exports and timestamp retention. Vite builds the actual MSAL client.

- Local HTTP delivery: a temporary loopback-only server served the SSO document and transformed MSAL entry successfully, then closed. This does not prove browser authentication or a tenant callback.

- Live AD lifecycle: **NOT VERIFIED**. Requires isolated Windows Server domain, runbook execution and before/after AD evidence.

- Live Entra SSO and break/fix: **NOT VERIFIED**. User has no lab tenant yet. App registration, successful sign-in, tenant log correlation, deliberate rejection and recovery remain gates.

- Privileged compromise handling: tabletop exercise only, using a synthetic 4728 field rendering.

Repeat the local checks

From the root: `powershell.exe -NoProfile -File .\tools\Test-Upgrade.ps1`.

Use a process-only policy override on your personal lab machine only if required and consistent with its controls. The runner produces dated JSON results under evidence/verification. Treat that file as local execution evidence, not proof of live directory or tenant behavior.

Failures found and corrected

- PowerShell 5.1 ConvertFrom-Json array wrapping initially combined certification identities. A regression test now verifies two distinct findings.

- Duplicate HR rows could conceal a leaver; input is now rejected before export.

- WhatIf initially suppressed evidence-folder creation; only intended local evidence writes explicitly bypass WhatIf while AD writes remain disabled.

- Activation failure logging could prevent containment; containment now runs independently and a test covers logging loss after enable.

- npm-generated Windows executable shims did not tolerate the ampersand in this project's path; scripts now invoke Vite through its relative Node entry point.

- Restoring browser event history initially regenerated timestamps; sanitized restoration now preserves the original UTC timestamp.

- Windows PowerShell evaluated script-relative default output paths before PSScriptRoot was available during parameter binding; defaults now resolve inside the script body. The preview test runs a copied project with default output paths.

Do not publish raw dependency logs, raw AD snapshots, or authentication traces. The source bundle omits live configuration and unreviewed evidence by design; manually inspect anything you share.